

# AI Solution Design

## Section A: Problem Decomposition

### 1. Supported User Question Taxonomy and Analytical Mapping

Production-grade systems should not treat incoming user queries as arbitrary text strings. Instead, the conversational engine passes natural language inputs through an **Intent-Based Classification Pipeline**. This pipeline dynamically splits the query:

1. The Functional Business Domain: The specific operational and dataset group targeted
2. The Technical Complexity Archetype: The structural and computational pattern required by the underlying query engine (Single-Fact Lookup, Cross-Domain Join).

### Taxonomy & Functional Patterns

Archetype A: Descriptive Lookup (Single-Fact Retrieval)

Requires explicit entity matching across structured attributes (Product, Region, Time) to extract a standalone data point. **Examples:** *"What were the sales of Diet Cola in the South region last week?"* | *"What is the current stock depletion rate for Carbonated Drinks in the West zone?"*

Archetype B: Comparative Analysis (Multi-Period / Multi-Entity Deltas)

Requires time-series manipulation, window functions and mathematical delta calculations. **Examples:** *"Did last month's campaign improve sales in the South region?"* | *"Which region had the highest week-on-week growth last month?"*

Archetype C: Diagnostic & Pivot Queries (Multi-Table Joins & Rankings)

High-complexity operations that span multiple transactional boundaries. The engine must navigate complex entity-relationship graphs joining fact tables (Sales, Daily Inventory Snapshots) with dimension tables (Promotional Calendars, SKU Hierarchies) and execute sorting operations. **Examples:** *"Which beverage categories saw inventory reduction during the promotion week?"* | *"Which product categories benefited most from the last trade promotion?"*

Archetype D: Ambiguous & Volatile Requests

Inputs lacking critical parameters (missing metrics, timeframes or product bounds). Rather than allowing the LLM to guess parameters which leads to silent data misinterpretations - the system triggers a hard conversational boundary or utilizes semantic defaults. **Examples:** *"How did the promotion do?"* → System Action: Halts execution and triggers a UI clarifying loop: *"Would you like to view the revenue impact, inventory depletion, or regional sales comparison for the active 'Summer Chill' campaign?"*

### 2. Structured Analytical Output Specification

- For each question type, the assistant produces a layered response:
- Narrative Summary: A 2–3 sentence plain-English answer to the question.
- Key Metric Highlight: The primary number (xy% revenue uplift) prominently surfaced.
- Comparison Context: Baseline vs promo period or region vs region.
- Chart Recommendation: A suggested visualization type (bar chart, trend line, heatmap).
- Explainability Trace: The SQL query used so analysts can validate the logic.

## Section B - AI Solution Approach

### 1. Methods Used

#### 1.1 Retrieval-Augmented Generation (RAG) over a Semantic Metadata Layer

Rather than feeding the raw database schema directly into the LLM, the system first builds a **Semantic Layer** - a human-readable catalog of every table, column and business metric definition. This catalog is **vectorized** (converted to numerical embeddings) and stored in a vector database. When a user asks a question, the system performs a **semantic search** against this catalog to retrieve only the most relevant tables and their column definitions.

#### 1.2 Text-to-SQL (NL-to-SQL) with Few-Shot Prompting

The core analytical engine converts natural language into executable SQL through a **structured, context-injected prompt** - not a raw open-ended LLM call. The LLM receives four precisely assembled components per query:

- System Role - constrains model to write only SELECT statements against Snowflake.
- Schema Context - most relevant tables + column descriptions retrieved by the Schema Retrieval Agent.
- Business Metric Definitions - locked formulas injected as rules
- Initial Shot - curated {natural language question → verified SQL} pairs specific to the dataset, teaching the model the exact join patterns, date filter conventions and aggregation level expected

The model is called at **temperature = 0** to minimize variance, as SQL generation is a precision task. Additionally, as the system scales and encounters highly complex edge-case queries where the architecture supports a future phase to **fine-tune** a smaller, task-specific model on our proprietary SQL dialect.

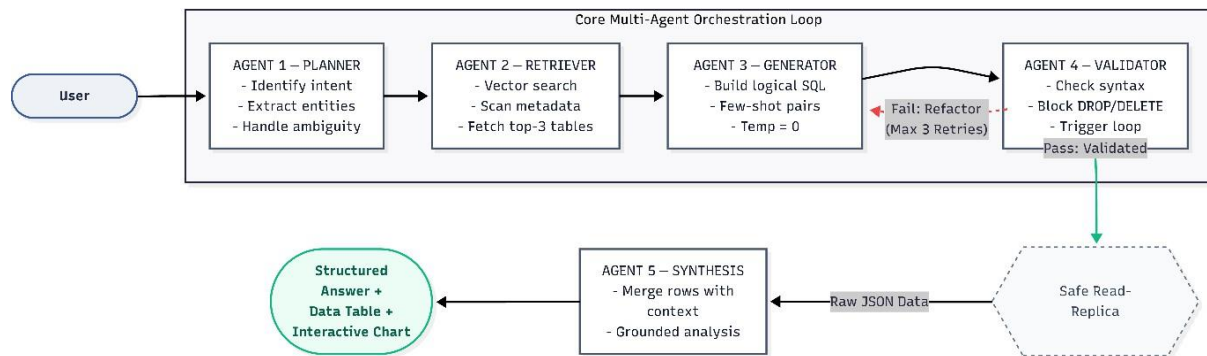
**1.3 Multi-Agent Orchestration with Self-Correction:** The system is orchestrated as a stateful multi-agent pipeline using LangGraph. Instead of a single monolithic LLM call, five specialized agents handle distinct stages of reasoning, each doing one job.

### 2. Why This Approach Is Suitable

| Challenge                                   | Why This Approach Addresses It                                                       |
|---------------------------------------------|--------------------------------------------------------------------------------------|
| LLMs don't know the organisation's database | RAG + Semantic Layer retrieves grounded, real schema context                         |
| LLMs can hallucinate metric formulas        | Pre-defined metric definitions are injected, not inferred                            |
| Complex multi-table queries are error-prone | A dedicated Validator Agent catches and self-corrects SQL errors before execution    |
| Business users ask ambiguous questions      | A Planner Agent decomposes and clarifies intent before SQL is written                |
| Data must be accurate, not just plausible   | Deterministic SQL execution against certified Layer data - no generative fabrication |

### 3. The Agentic Construct

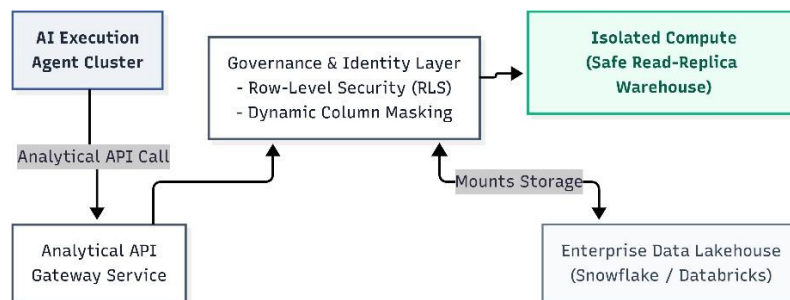
The system implements the following **5-agent pipeline**, orchestrated as a directed state graph:



## Section C - Data Interaction Strategy

### 1. How the AI System Accesses Analytical Datasets

#### 1.1 Distributed Lakehouse Architecture with Zero-Trust Isolation



The system should be built on a modern distributed Lakehouse architecture (Snowflake Managed Governance or Databricks Unity Catalog) with a fundamental design principle. The AI execution environment is completely separated from core data resources - preventing performance degradation, data corruption and unauthorized access.

**1.2 Governance and Identity Layer:** User identities are bound to the session via **OAuth2 tokens** passed through from the enterprise application. When the agent acts on behalf of a user, the data platform's governance engine (Unity Catalog / Snowflake RBAC) dynamically applies security rules in real time:

- **Row-Level Security (RLS):** A regional manager scoped to the 'X' zone will only see rows where `region_id = 'X'` - enforced at the platform level.
- **Dynamic Column Masking:** Sensitive financial columns (cost-to-serve, margin equations) are automatically obfuscated for users without explicit clearance - the agent receives a masked view, not the raw value.

### 2. How the System Performs Aggregations

The runtime environment should enforce a strict policy: all aggregation, grouping and metric computation executes natively within the cloud data platform - never locally.

Aggregations can operate at two levels. **High-frequency business metrics** - weekly promo uplift by region and **Monthly sales by category** are pre-materialized as curated tables during the nightly dbt pipeline run.

Forbidden operations (DROP, DELETE, INSERT) are rejected and unaggregated row-level reads without filters are blocked. All column references are validated against the live schema catalog.

## Section D - Limitations & Risks

The LLM may generate SQL that is syntactically valid but semantically incorrect - for example, joining two tables on the wrong key or applying a date filter to the wrong column. Additionally, as with any generative system interacting with a database, there is an inherent risk of **Prompt Injection**, where a user attempts to trick the model into revealing unauthorized schema details or executing resource-heavy commands.

Mitigation Strategy (3-layer defense):

1. **Semantic Layer Grounding:** Every LLM prompt includes explicit, pre-defined formulas for all business metrics. The model cannot invent a formula for 'Promotional uplift' where the definition is given. This eliminates the largest class of semantic errors.
2. **Critic Agent Validation:** Before any SQL is executed, the Validator Agent checks column existence against the live schema, verifies join keys match documented relationships and confirms the aggregation level matches the question's granularity
3. **Result Plausibility Check + Transparency:** After execution, the Synthesis Agent includes the exact SQL used in its response. Business users can optionally review the query and data analysts can validate it before executive reporting. For high-stakes outputs, a **Human-in-the-Loop** approval gate is activated.

## Section E - Design Trade-Off

### Trade-Off: Accuracy Over Latency

The system routes every new user question through a full **5-agent pipeline** (approximately 5–8 seconds end-to-end) rather than a faster single-shot LLM call.

**Why This Trade-Off Was Made:** In a business context where promotional decisions or inventory actions are taken based on the AI's output, a fast but wrong answer is worse than a slightly slower but improved output. The multi-agent design - with a dedicated Planner, Schema Retriever, Generator, Validator and Synthesis agent dramatically reduces the hallucination rate compared to a single-prompt approach.

### How Latency Is Managed Without Sacrificing Accuracy:

- **Semantic Caching:** Reuses past answers for similar questions (cosine similarity > 0.95) to reduce latency (<100ms) and save API costs.
- **Model Routing:** Uses smaller, cheaper LLMs for basic tasks (routing/validation) and reserves expensive, heavy models strictly for complex SQL generation to control token costs.

This is the same trade-off made by Google's Duet AI for BigQuery and Microsoft's Copilot for Power BI - both prioritize correctness and traceability over raw response speed, then use caching and pre-computation to recover perceived performance.